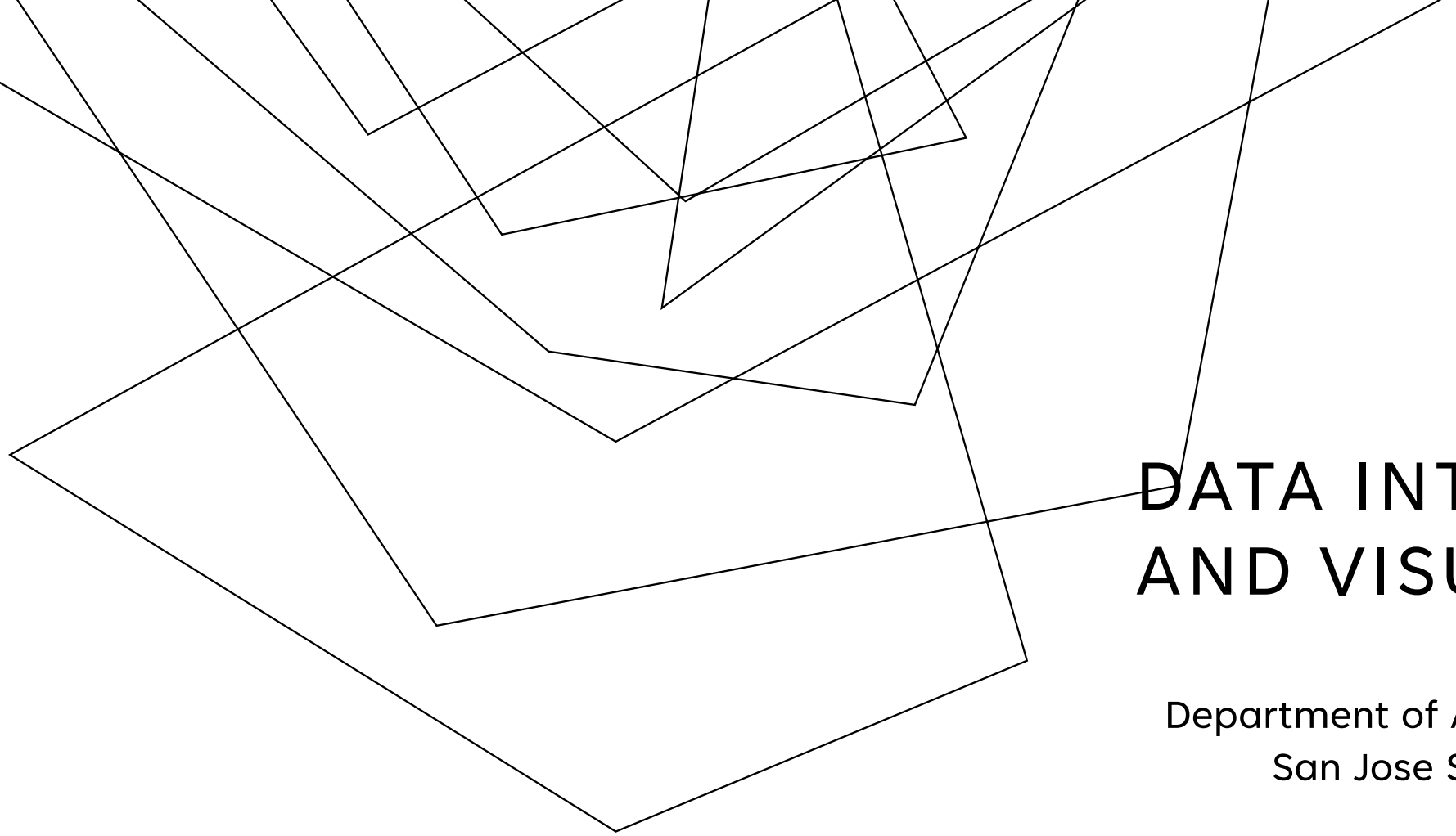


# DATA INTELLIGENCE AND VISUALIZATION

Department of Applied Data Science
San Jose State University

Instructor: Dr. Guannan Liu

September 11

Fall 2026

# TODAY'S LECTURE

- **Review Last Session**
- EDA Task

# In-Class Practice: 5 Bonus Points

- Download the Airbnb data **("listings.csv") for Albany**, New York, United States from: https://insideairbnb.com/get-the-data/
- Part 1: Present the geographical data using "latitude", "longitude" and "price" with Matplotlib.
- Part 2: Present the same data with Plotly.
- How you encode the three variables is your choice. State one sentence question your figure answers.
- Submit one .ipynb file on Canvas that includes both Part 1 and Part 2.

# In-Class Practice: Matplotlib

## In-class Task 2

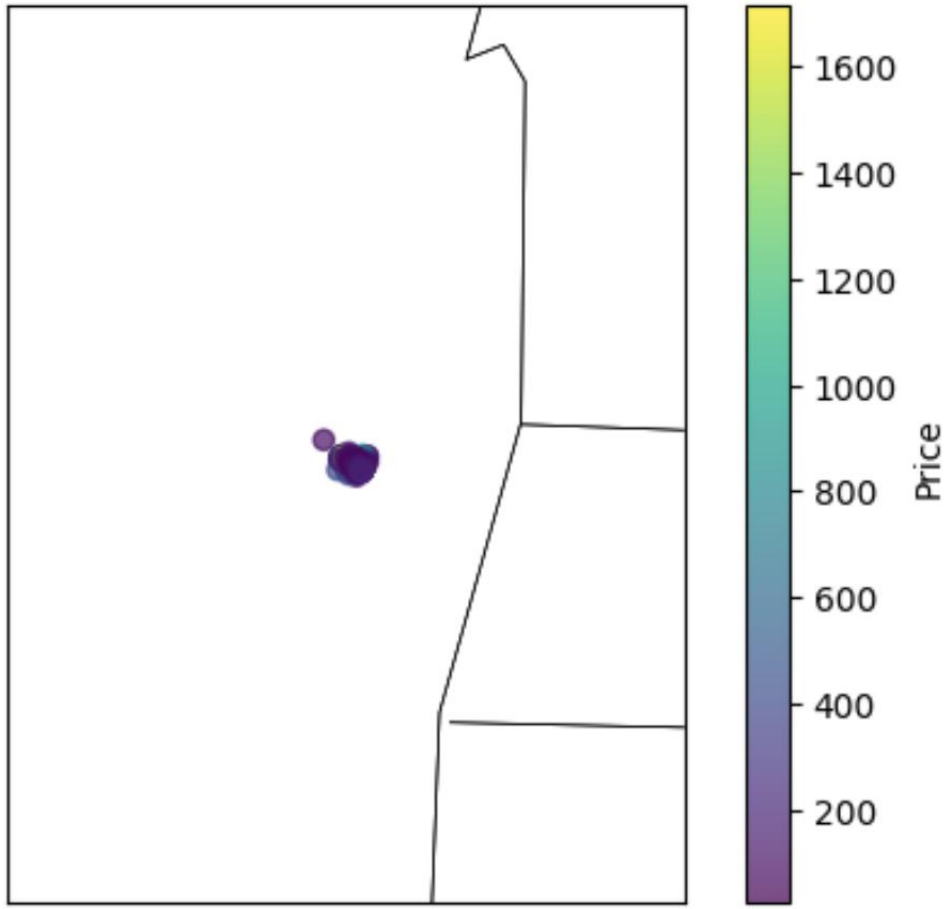
```
from mpl_toolkits.basemap import Basemap
```

```
df = pd.read_csv('listings.csv')
df = df.dropna(subset=['price'])
plt.figure()
m = Basemap(projection='merc', llcrnrlat=df['latitude'].min()-1,
            urcrnrlat=df['latitude'].max()+1,
            llcrnrlon=df['longitude'].min()-1,
            urcrnrlon=df['longitude'].max()+1, resolution='l')

m.drawcoastlines()
m.drawcountries()
m.drawstates()

x, y = m(df['longitude'].values, df['latitude'].values)
sc = m.scatter(x, y, c=df['price'], cmap='viridis', alpha=0.7, marker='o')
plt.colorbar(sc, label='Price')

plt.show()
```




Ref:
https://matplotlib.org/basemap/stable/users/examples.html

# In-Class Practice: Plotly

```
import plotly.express as px
```

```
# Scatter map plot using Plotly
fig = px.scatter_mapbox(df, lat='latitude', lon='longitude', color='price',
                        size='price', hover_name='price',
                        mapbox_style="open-street-map",
                        color_continuous_scale=px.colors.sequential.Viridis,
                        zoom=10)

# Show the interactive plot
fig.show()
```




Ref:
https://plotly.com/python/

https://plotly.github.io/plotly.py-docs/index.html

# TODAY'S LECTURE

- Review Last Session
- **EDA Task**

# Group Quiz 2-Q1: EDA Report (15 Points)

- Work in your project groups. One submission per group, Do **NOT** put group number or group member name on your .ipynb file.
- Data: used_cars_data.csv (Canvas)
- Goal: predict the resale price
- Task: Conduct a **complete** exploratory data analysis (EDA) of the dataset
- Time: 40 minutes.
- Submit: One .ipynb file on Canvas.

- Hint: A complete EDA report is one the other developer could start building a model from, every decision already made in your notebook, without asking the authors anything.

# Group Quiz 2-Q2: Peer Evaluation (5 points)

- You can discuss in group. You receive another group's notebook, with the group name removed.
- Everyone need to fill the Peer Evaluation Form and submit in class: 20 minutes.
- **Your 5 evaluation points come from the quality of your evaluation**

- **Grade: 30% peer evaluation + 70% ISA grade.**
- **Your scores set 30% of the other group's Q1 grade. Be serious about it.**

***Break***: 10 minutes

# Tonight's Dataset: Cars4U

- **Used-car listings from India**
  - One row is one car offered for resale.
  - Price is quoted in Lakh (1 Lakh = 100,000 rupees; 1 Crore = 100 Lakh).
- **The task you are given**
  - Predict the resale price of a used car.
- **Raw records looks like this**

```
[78]: df.head()
```

[78]:

| | S.No. | Name | Location | Year | Kilometers_Driven | Fuel_Type | Transmission | Owner_Type | Mileage | Engine | Power | Seats | New_Price | Price |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | Maruti Wagon R LXI CNG | Mumbai | 2010 | 72000 | CNG | Manual | First | 26.6 km/kg | 998 CC | 58.16 bhp | 5.0 | NaN | 1.75 |
| 1 | 1 | Hyundai Creta 1.6 CRDi SX Option | Pune | 2015 | 41000 | Diesel | Manual | First | 19.67 kmpl | 1582 CC | 126.2 bhp | 5.0 | NaN | 12.50 |
| 2 | 2 | Honda Jazz V | Chennai | 2011 | 46000 | Petrol | Manual | First | 18.2 kmpl | 1199 CC | 88.7 bhp | 5.0 | 8.61 Lakh | 4.50 |
| 3 | 3 | Maruti Ertiga VDI | Chennai | 2012 | 87000 | Diesel | Manual | First | 20.77 kmpl | 1248 CC | 88.76 bhp | 7.0 | NaN | 6.00 |
| 4 | 4 | Audi A4 New 2.0 TDI Multitronic | Coimbatore | 2013 | 40670 | Diesel | Automatic | Second | 15.2 kmpl | 1968 CC | 140.8 bhp | 5.0 | NaN | 17.74 |

# The Shape of an EDA

Structure – Clean – Univariate – Bivariate – Multivariate – Decisions

| Act | Question it answers | Charts |
|---|---|---|
| 1 Structure and cleaning | What is actually in this file, and what is broken? | console output, missing-rate bar |
| 2 Univariate — categorical | What kind of cars are these? | bar, grouped bar, dot plot, heatmap |
| 3 Univariate — numeric | What shape is each variable, and the target above all? | histogram, density, box, violin, QQ |
| 4 Bivariate | What moves with price? | scatter, correlation, box by group |
| 5 Multivariate | What is really driving it, once you hold something constant? | heatmap, pairplot, faceted scatter |

The sixth stage produces a **table of decisions** is what the next person on your project actually needs.

# How Do You Know an EDA Is Finished?

- **An EDA ends when you can make the decisions the next step needs.**
  - A complete EDA report is one a developer could start building a model from, every decision already made in your notebook, without asking the authors anything.
- **Three stopping criteria**
  - **1. No structural unknowns left.** Every column has a stated type, unit, missing rate and verdict.
  - **2. The target and features are fully understood.** Shape, skew, outliers, missingness, and whether it needs a transform.
  - **3. The relationships that matter are located.** Strong predictors, confounders, leakage.

# Structure and Cleaning

Structure – Clean – Univariate – Bivariate – Multivariate – Decisions

- **Every EDA starts the same way**
  - df = pd.read_csv('used_cars_data.csv')
  - df.head(5)/df.tail(5) — what does one record look like?
  - df.shape — 7,253 rows x 14 columns
  - df.info() — types and non-null counts
  - df.describe() — the five-number summary of every numeric column

# Assignment2 Solu: EDA Phase

## ➢ Understand the Data Structure:

```
[78]: df.head()
```

[78]:

| | S.No. | Name | Location | Year | Kilometers_Driven | Fuel_Type | Transmission | Owner_Type | Mileage | Engine | Power | Seats | New_Price | Price |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | Maruti Wagon R LXI CNG | Mumbai | 2010 | 72000 | CNG | Manual | First | 26.6 km/kg | 998 CC | 58.16 bhp | 5.0 | NaN | 1.75 |
| 1 | 1 | Hyundai Creta 1.6 CRDi SX Option | Pune | 2015 | 41000 | Diesel | Manual | First | 19.67 kmpl | 1582 CC | 126.2 bhp | 5.0 | NaN | 12.50 |
| 2 | 2 | Honda Jazz V | Chennai | 2011 | 46000 | Petrol | Manual | First | 18.2 kmpl | 1199 CC | 88.7 bhp | 5.0 | 8.61 Lakh | 4.50 |
| 3 | 3 | Maruti Ertiga VDI | Chennai | 2012 | 87000 | Diesel | Manual | First | 20.77 kmpl | 1248 CC | 88.76 bhp | 7.0 | NaN | 6.00 |
| 4 | 4 | Audi A4 New 2.0 TDI Multitronic | Coimbatore | 2013 | 40670 | Diesel | Automatic | Second | 15.2 kmpl | 1968 CC | 140.8 bhp | 5.0 | NaN | 17.74 |

```
[79]: df.tail()
```

[79]:

| | S.No. | Name | Location | Year | Kilometers_Driven | Fuel_Type | Transmission | Owner_Type | Mileage | Engine | Power | Seats | New_Price | Price |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7248 | 7248 | Volkswagen Vento Diesel Trendline | Hyderabad | 2011 | 89411 | Diesel | Manual | First | 20.54 kmpl | 1598 CC | 103.6 bhp | 5.0 | NaN | NaN |
| 7249 | 7249 | Volkswagen Polo GT TSI | Mumbai | 2015 | 59000 | Petrol | Automatic | First | 17.21 kmpl | 1197 CC | 103.6 bhp | 5.0 | NaN | NaN |
| 7250 | 7250 | Nissan Micra Diesel XV | Kolkata | 2012 | 28000 | Diesel | Manual | First | 23.08 kmpl | 1461 CC | 63.1 bhp | 5.0 | NaN | NaN |
| 7251 | 7251 | Volkswagen Polo GT TSI | Pune | 2013 | 52262 | Petrol | Automatic | Third | 17.2 kmpl | 1197 CC | 103.6 bhp | 5.0 | NaN | NaN |
| 7252 | 7252 | Mercedes-Benz E-Class 2009-2013 E 220 CDI Avan... | Kochi | 2014 | 72443 | Diesel | Automatic | First | 10.0 kmpl | 2148 CC | 170 bhp | 5.0 | NaN | NaN |

# Observations & Issues

- **Missing values: New_Price** and **Price** columns contain many NaN (missing)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7253 entries, 0 to 7252
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   S.No.              7253 non-null   int64
 1   Name               7253 non-null   object
 2   Location           7253 non-null   object
 3   Year               7253 non-null   int64
 4   Kilometers_Driven  7253 non-null   int64
 5   Fuel_Type          7253 non-null   object
 6   Transmission       7253 non-null   object
 7   Owner_Type         7253 non-null   object
 8   Mileage            7251 non-null   object
 9   Engine             7207 non-null   object
 10  Power              7207 non-null   object
 11  Seats              7200 non-null   float64
 12  New_Price          1006 non-null   object
 13  Price              6019 non-null   float64
dtypes: float64(2), int64(3), object(9)
memory usage: 793.4+ KB
```

```
missing_percent = (df.isnull().sum()/(len(df)))*100
print(missing_percent)
```

```
S.No.                0.000000
Name                 0.000000
Location             0.000000
Year                 0.000000
Kilometers_Driven    0.000000
Fuel_Type            0.000000
Transmission         0.000000
Owner_Type           0.000000
Mileage              0.027575
Engine               0.634220
Power                0.634220
Seats                0.730732
New_Price           86.129877
Price               17.013650
dtype: float64
```

# Trap 1 — describe() Quietly Drops Three Columns

```
df.describe().T
```

| | count | mean | std | min | max |
|---|---|---|---|---|---|
| S.No. | 7253.00 | 3626.00 | 2093.91 | 0.00 | 7252.00 |
| Year | 7253.00 | 2013.37 | 3.25 | 1996.00 | 2019.00 |
| Kilometers_Driven | 7253.00 | 58699.06 | 84427.72 | 171.00 | 6500000.00 |
| Seats | 7200.00 | 5.28 | 0.81 | 0.00 | 10.00 |
| Price | 6019.00 | 9.48 | 11.19 | 0.44 | 160.00 |

*5 rows. Where are Mileage, Engine and Power?*

```
df.dtypes
```

| | |
|---|---|
| S.No. | int64 |
| Name | str |
| Location | str |
| Year | int64 |
| Kilometers_Driven | int64 |
| Fuel_Type | str |
| Transmission | str |
| Owner_Type | str |
| **Mileage** | **str** |
| **Engine** | **str** |
| **Power** | **str** |
| Seats | float64 |
| New_Price | str |
| Price | float64 |

*stored as text, so describe() ignores them*

**Rule: the number of rows in describe() must match the number of columns you believe are numeric.**

# Four Problems, Found in Five Minutes

| # | Problem | Columns | Why it matters |
|---|---|---|---|
| 1 | **Redundant structure** | S.No. | A row number. It will show up in your correlation matrix. |
| 2 | **Numbers stored as text** | Mileage, Engine, Power, New_Price | describe() skips all four. Two of them mix units, and they mix them in two different ways. |
| 3 | **Missing data** | New_Price (86%), Price (17%) | One is nearly empty. The other is the target. |
| 4 | **Packed / indirect fields** | Name, Year | Name holds brand **model** variant. Year is not depreciation. |

# Fix 1: Drop the Index Column

- **Issue**
  - S.No. is a serial number copied from the row index. It carries no information.
- **Why it is worth a slide**
  - It enters describe(), and it enters the correlation matrix, looking like a feature.
  - **And it silently disables duplicate detection:** see three slides from now.
- **Fix**
  - df.drop(columns=['S.No.'], inplace=True)
  - df.reset_index(drop=True, inplace=True)

# How It Affects: Duplicates

- **The same file, the same function, two different answers.**

```
df.duplicated().sum()
0

df.drop(columns=['S.No.']).duplicated().sum()
1
```

*a unique ID makes duplicated() useless — drop it first*

Drop the ID column before you check for duplicates: otherwise the check cannot ever fail.

# Fix 2a: Take Inventory of the Units

- Before converting anything, find out what units are actually present.

```
units = {}
for col in ["Mileage", "Engine", "Power", "New_Price"]:
    suffix = df[col].dropna().astype(str).str.extract(r"([A-Za-z/]+)\s*$")[0]
    units[col] = suffix.value_counts().to_dict()

for col, counts in units.items():
    print(f"{col:<12s} {counts}")
```

```
Mileage      {'kmpl': 7177, 'km/kg': 74}
Engine       {'CC': 7207}
Power        {'bhp': 7207}
New_Price    {'Lakh': 986, 'Cr': 20}
```

| Column | Units present | What is mixed | Can one constant fix it? |
|---|---|---|---|
| `Mileage` | `kmpl` (7,177) · `km/kg` (74) | two different **physical quantities** — per litre vs per kilogram | **Only via a fuel property.** No definition converts a volume into a mass, so the factor depends on `Fuel_Type` — and on which property you pick. |
| `New_Price` | `Lakh` (986) · `Cr` (20) | two **scales** of the same quantity | **Yes, exactly.** 1 Cr = 100 Lakh is a definition, not an estimate. |

# Fix 2b: Mileage Mixes Two Physical Quantities

- kmpl = km per litre (7,177 rows) · km/kg = km per kilogram (74 rows)
- The unit is decided by Fuel_Type. Fuel_Type is in the data, so look up one factor per fuel instead of pretending 1 km/kg = 1 kmpl.
- But litre and kilogram only meet through a property of the fuel, and which property you pick decides the answer.

```
PETROL_MJ_PER_L = 33.1                  # published, and therefore arguable
FUEL_MJ_PER_KG  = {"CNG": 48.0, "LPG": 46.0}
KMPL_FACTOR     = {f: mj / PETROL_MJ_PER_L ...}

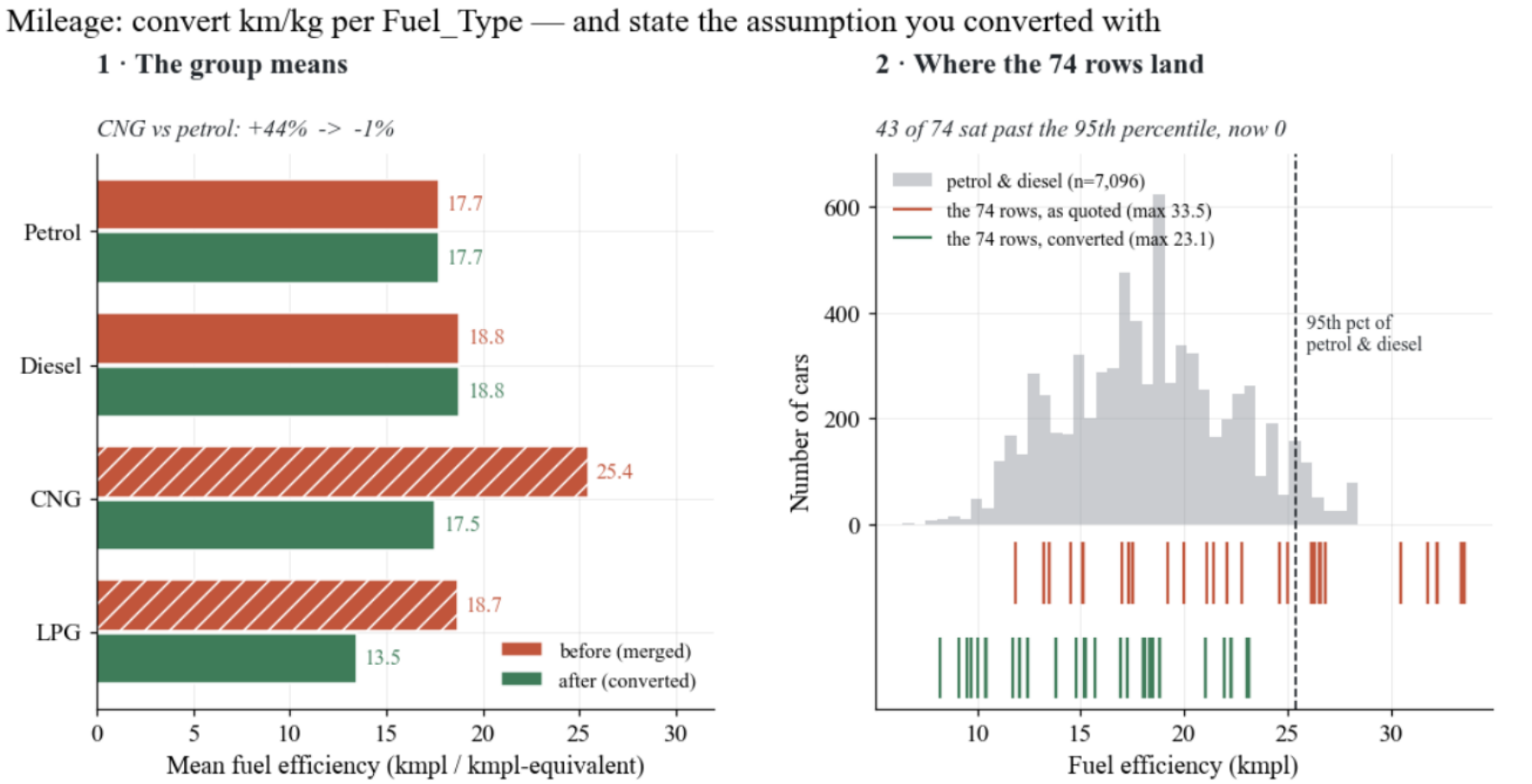
# the factor is a property of the fuel, so it is looked up per row
factor = df["Fuel_Type"].map(KMPL_FACTOR)
clean["Mileage"] = value.where(unit != "km/kg", value / factor)

CNG mean, as quoted   ->  25.4 km/kg    +44% vs petrol
CNG mean, converted   ->  17.5 kmpl-eq   -1% vs petrol
```

*the 44% gap was the denominator, and the conversion proves it*

Energy basis: 1 kg CNG carries the energy of 1.45 litres of petrol, so 25.4 km/kg becomes 17.5 kmpl-equivalent. The assumption lives in a named constant, where a reader can argue with it.

# Fix 2b — Before and After, and the Basis You Chose




Choosing the denominator is choosing the question. Convert if you can but **write down what you converted with.**

# Fix 2c: New_Price Mixes Two Scales

- 1 Crore = 100 Lakh. 986 rows are quoted in Lakh; 20 rows are quoted in Cr.
- Strip the suffix without converting, and those 20 rows become 100x too small
  - '1.2 Cr' -> naive: 1.2 correct: 120.0
- Nothing raises an error. The wrong numbers still look like plausible prices.
- Unlike Mileage, this one has a rescue: same quantity, two scales, so one multiplication puts the column back together.

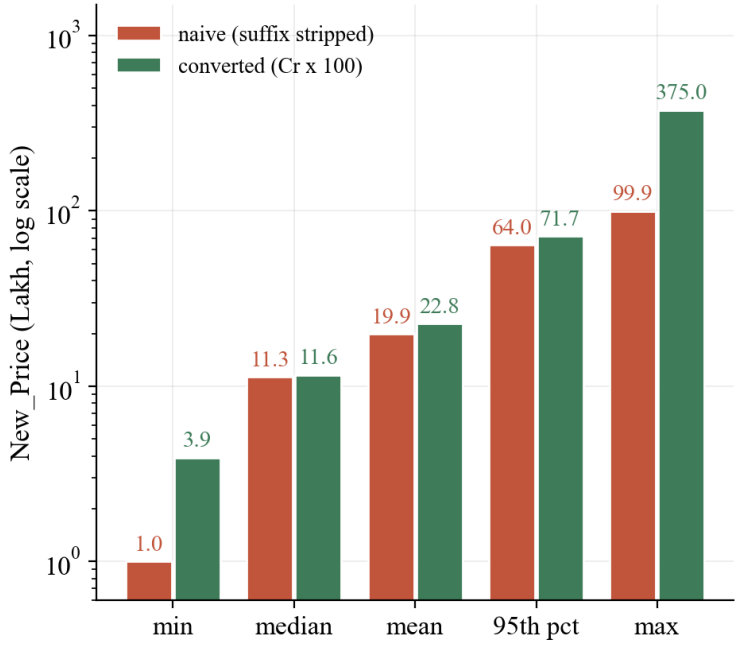
> You cannot clean a column you do not understand. Domain first, code second.

# Fix 2c: Before and After

New_Price: 1 Cr = 100 Lakh — one multiplication, and it decides which cars the model thinks are expensive

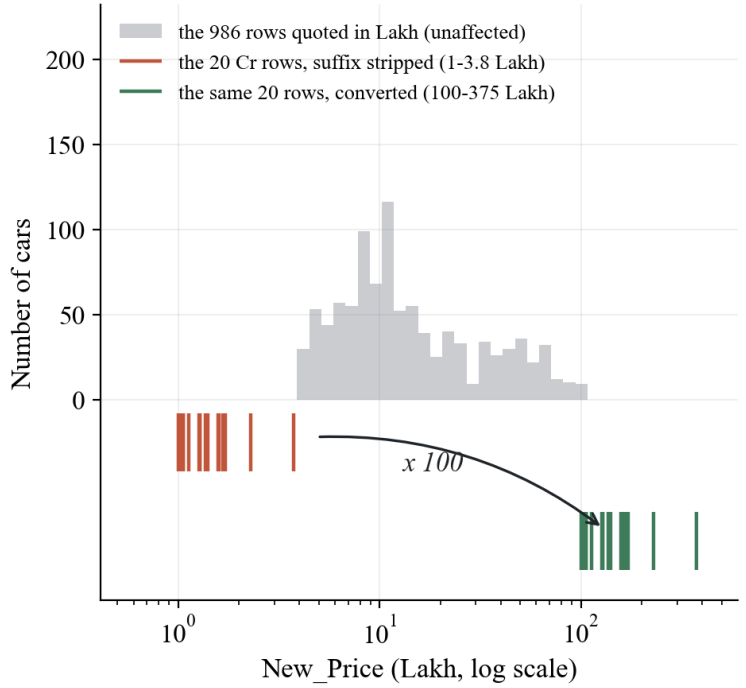
**1 · What describe() would show you**

*the median moves 0.30 Lakh (+3%) — only the max shouts*



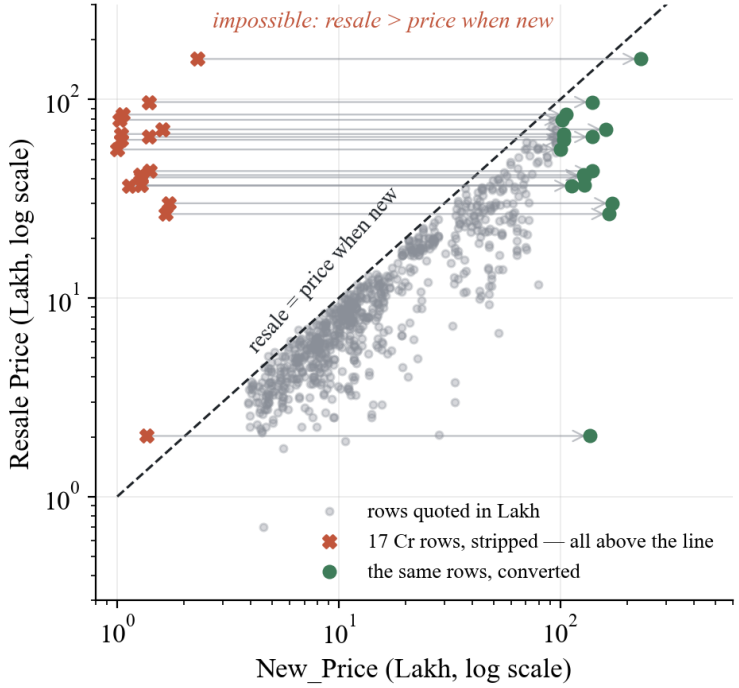

**2 · Where the 20 rows land**

*stripped, the 20 most expensive cars become the cheapest column entries*




**3 · The check that catches it without the domain**

*rows where resale > new: 20 -> 3*




**The summary statistics barely moved. The ranking turned upside down.** A used car cannot cost more than it did new — that check finds all 20 rows without knowing what a Crore is.

# Fix 2d: Convert, and Check What You Broke

- Mileage is converted onto one scale; Engine, Power and New_Price become numeric. describe() now returns 9 columns, not 5.

```
column               before     after
-------------------------------------------
S.No.                int64      int64
Name                 str        str
Location             str        str
Year                 int64      int64
Kilometers_Driven    int64      int64
Fuel_Type            str        str
Transmission         str        str
Owner_Type           str        str
Mileage              str        float64
Engine               str        float64
Power                str        float64
Seats                float64    float64
New_Price            str        float64
Price                float64    float64
Mileage_Unit         -          str   (new)
```

*describe() now returns 9 columns, not 5 — and Mileage is one unit throughout*

# Fix 3: Missing Values Come in Three Disguises

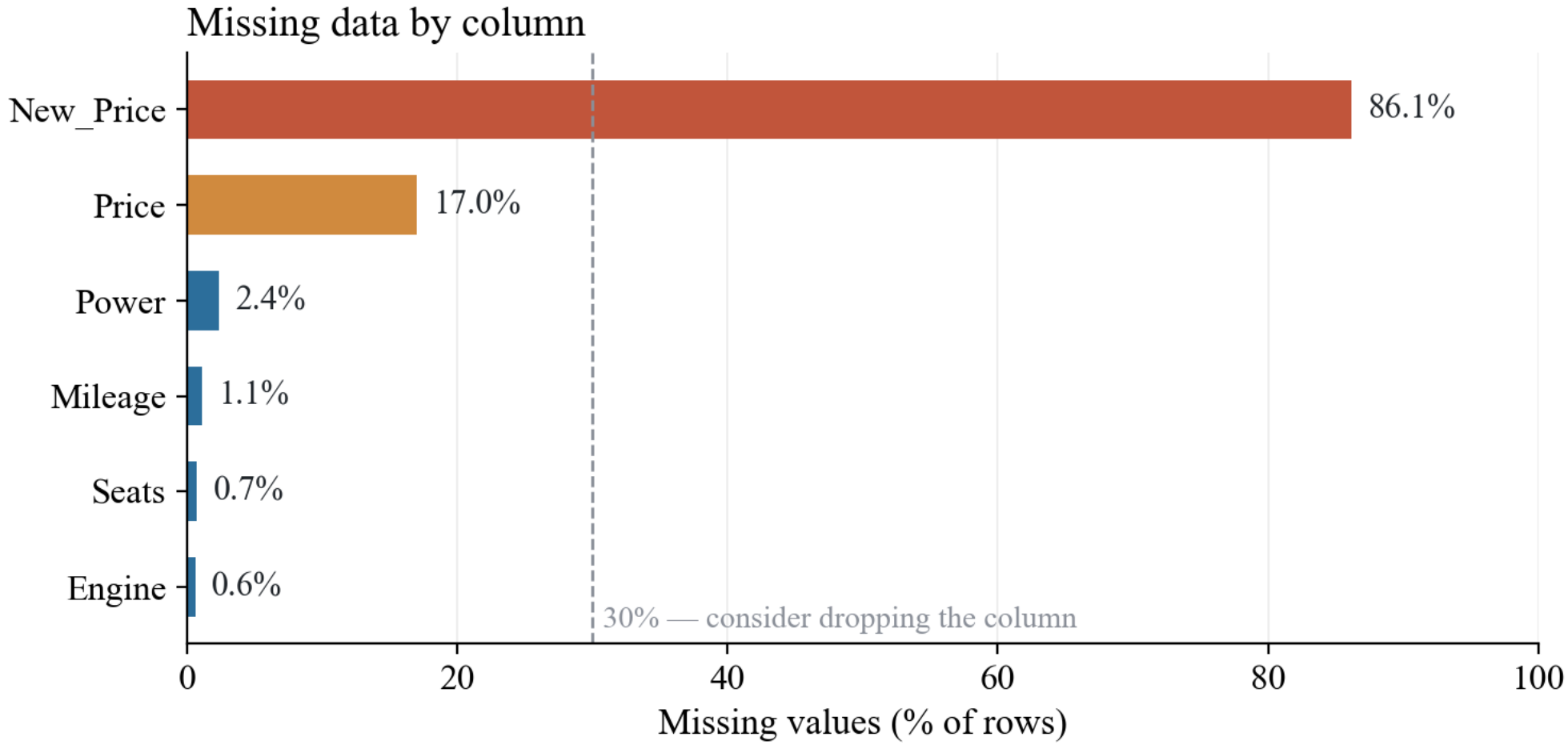
- **isna() answers exactly one question: was this value marked as missing?** Two of the three below were never marked.

| The disguise | How it is written | In Cars4U | Does isna() see it? |
|---|---|---|---|
| **1 · Marked as missing** | NaN · None · NULL | New_Price 86% · Price 17% · Seats, Engine, Power ~0.7% | **Yes — the only one it can see** |
| **2 · A string standing in for a number** | "null" · "" · "NA" · "N/A" · "-" · "?" · "unknown" | Power == "null bhp": 129 rows, the 5th most common value in the column | **No — the column is text, so every row counts as present** |
| **3 · A legal number that is impossible** | 0 · -1 · 999 · -999 · 9999 · 1900-01-01 | Mileage == 0: 81 rows · Seats == 0: 1 row | **No — and describe() averages them in** |

Never assume missing means NaN. Power reports 0.63% missing and is really 2.41%; Mileage reports 0.03% and is really 1.14%. Read a column's values — value_counts() on the raw column — before you trust any missing rate.

# The Missing-Data Picture

- **Two columns are a problem. They are problems of completely different kinds.**



# How Much Missing Is Too Much?

- **General approaches include:**
  - Remove rows or columns with excessive missing values.
  - Impute using mean, median, mode, or domain knowledge/ML-based imputation.

| Range | Typical Action | Explanation |
|---|---|---|
| < 5% missing | Drop missing values or Simple imputation (mean/median/mode) | Small impact, easy to fix |
| 5–30% missing | Impute carefully (imputation or domain knowledge) | Missingness could introduce bias; check distribution |
| > 30% missing | Consider dropping the column | Too much missing data; unreliable information |
| Target variable | Usually drop missing rows | Cannot train a model otherwise |

Price is 17% missing — the 'impute carefully' band. But Price is the TARGET, and you cannot impute a target: you would train on answers you invented.

# Mean or Median?

- **The classic argument**
  - Data: 10, 12, 11, 13, 100, NaN
  - mean = 29.2 (dragged by one value) median = 12 (unmoved)
- **Which means the table below cannot be finished yet**

| Column | Missing | Decision |
|---|---|---|
| Mileage, Engine, Power, Seats | 0.6 – 2.4% | ? mean or median, or drop rows |
| New_Price | 86.13% | drop the column |
| Price | 17.00% | drop the rows — it is the target |

# Fix 4: Derive What You Actually Want to Model

- **Year is not depreciation**
  - CURRENT_YEAR = 2026
  - df[‘Car_Age’] = CURRENT_YEAR - df[‘Year’] → range 7 to 30 years
- **Name is three fields in a trench coat**
  - 'Maruti Wagon R LXI CNG' = brand model variant, and there are 2,041 of them
  - df[‘Brand’] = df[‘Name’].str.split().str[0] → 33 brands
- **One column with 2,041 values is unusable. One with 33 is a feature.**

# The Output of an EDA Is a Decision Table

Structure – Clean – Univariate – Bivariate – Multivariate – Decisions

| Column / issue | Decision | Because |
|---|---|---|
| S.No. | drop the column | row number; it also disabled duplicated() |
| Mileage, Engine, Power | strip units, cast to float | describe() was silently skipping them |
| New_Price rows in 'Cr' | convert Cr to Lakh (x100) | 1 Cr = 100 Lakh; 20 rows |
| Power == 'null bhp' | treat as missing | 129 rows; true missing rate 0.6% -> 2.4% |
| Mileage == 0 / Seats == 0 | treat as missing | 81 rows and 1 row; both impossible |
| duplicate row | drop | only visible once S.No. was removed |
| New_Price | drop the column | 86% missing |
| Price | drop rows where missing | it is the target; never impute a target |

# Univariate: Categorical Variables

Structure – Clean – **Univariate** – Bivariate – Multivariate – Decisions

## ➢ ACT 2

- **What kind of cars are in this file?**
- A counting question, so the answer is a bar, a dot plot or a heatmap.
- Week 2: a chart type is an answer. You cannot choose it before you have the question.

# Directory of Visualizations: Amounts



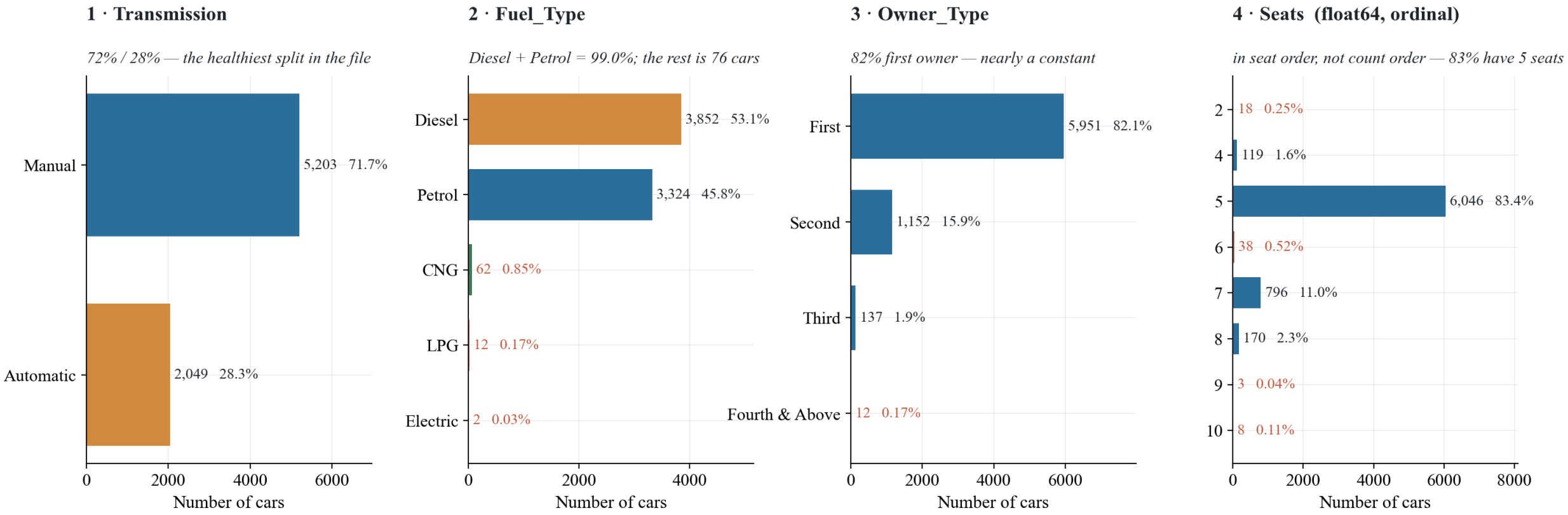

Source: Fundamentals of Data Visualization,
Claus O. Wilke, O'Reilly Media 2019

# Write the Question First, Then Pick the Chart

- **Week 2: a chart type is an answer. You cannot choose it before you have the question.**
- **Tonight's question for this act**
  - "What kind of cars are in this file?"
- **That is an amounts question — a count per category.**
  - Not a distribution question (that is Act 3).
  - Not a relationship question (that is Act 4).
- **Amounts have four standard answers: bar, grouped bar, dot plot, heatmap.**

# Univariate Categoricals: the Four Small Columns

Four columns with a handful of categories — sorted by count, except the one that has an order of its own



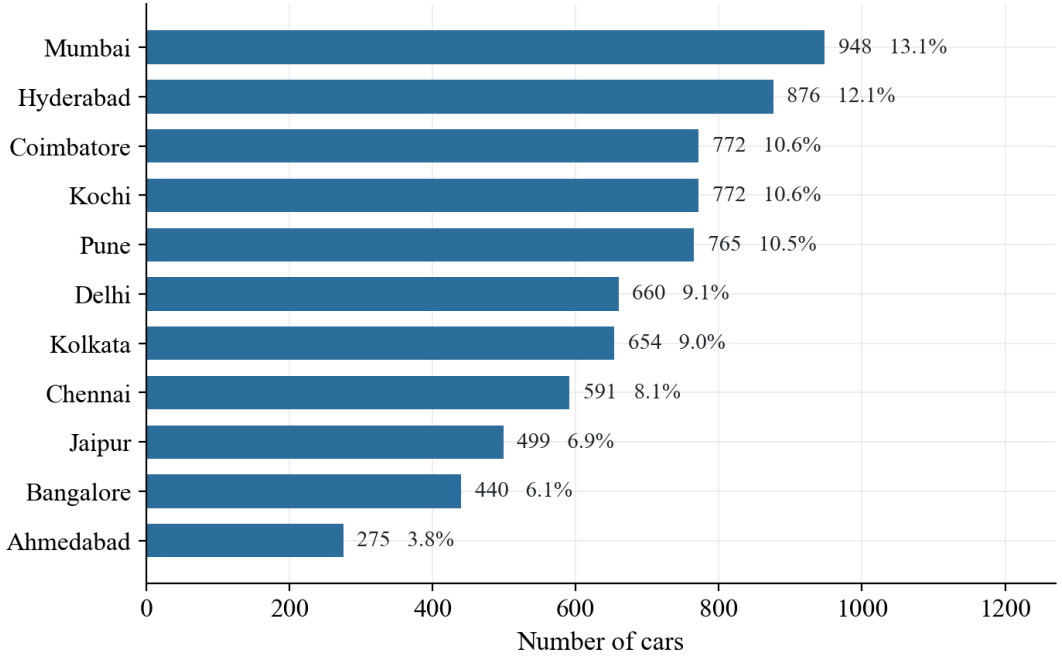

**Seats is a categorical wearing a float64.** Eight distinct values, 83% of them the same one.

# Univariate Categoricals: Location and Brand

The two columns with real category counts — same question, and the long tail is part of the answer

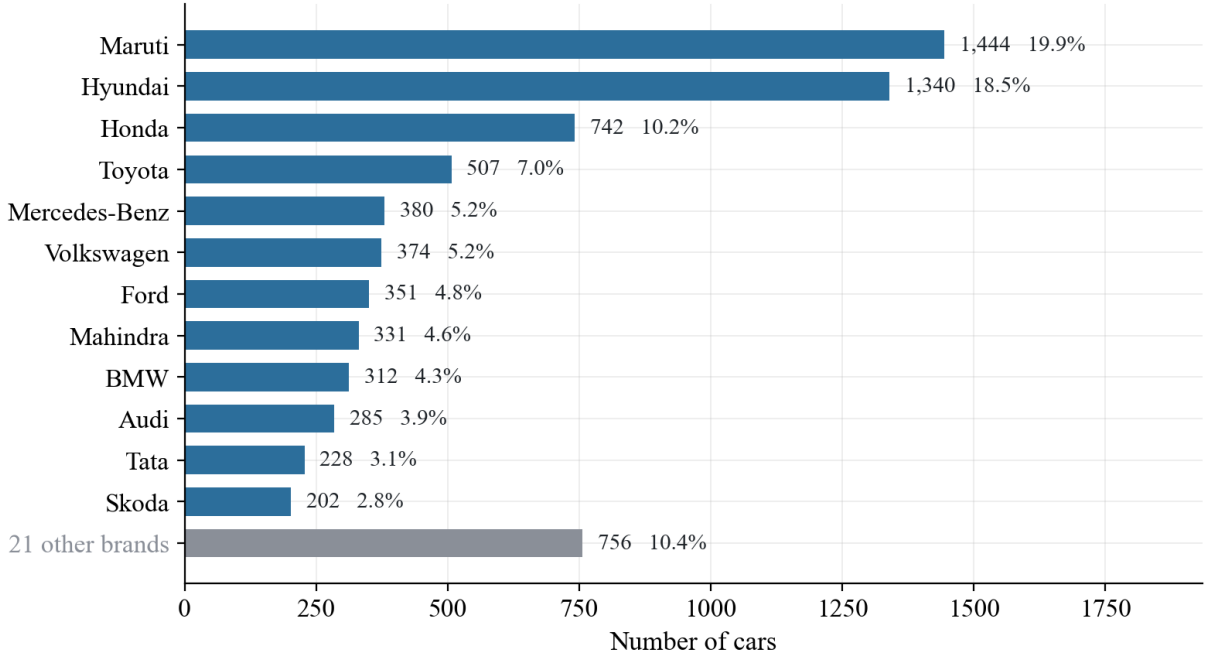
**1 · Location**

*11 cities, the largest only 13% — the flattest column in the file*




**2 · Brand (derived from Name in Act 1)**

*33 brands; the top 5 are 61% of the file, and 13 of them have under 30 cars each*



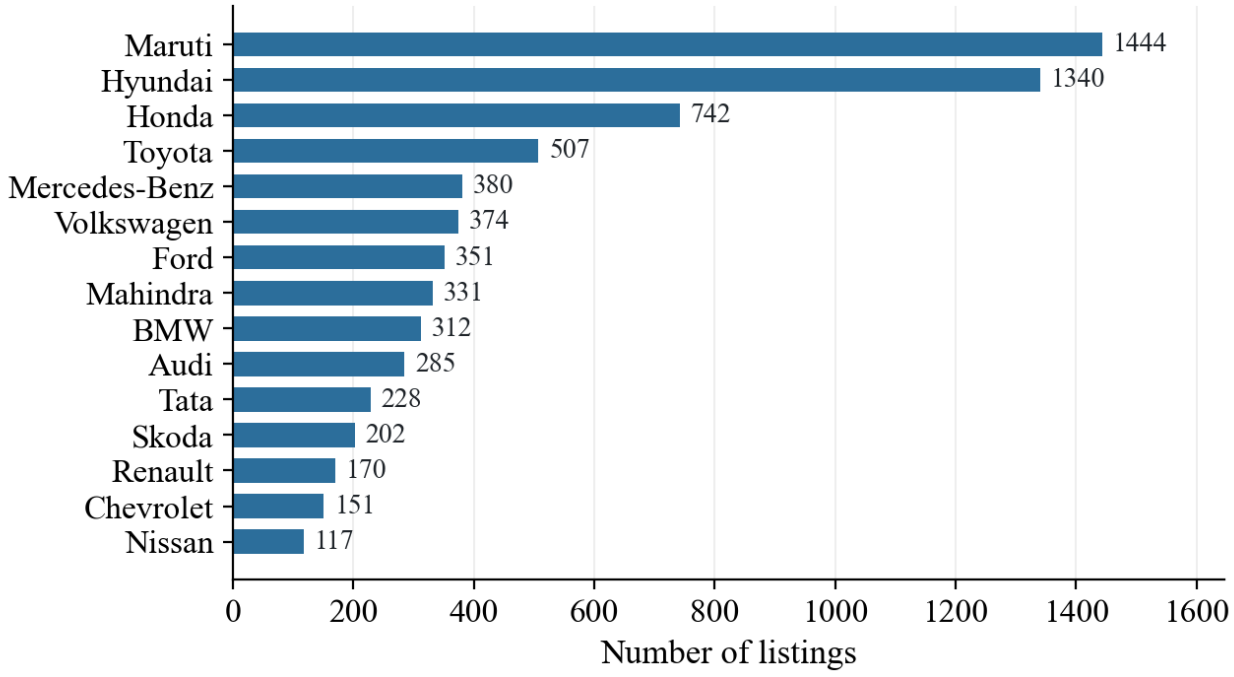

**Location is flat; Brand is a long tail.**

# Bar Charts: Three Decisions, Not One

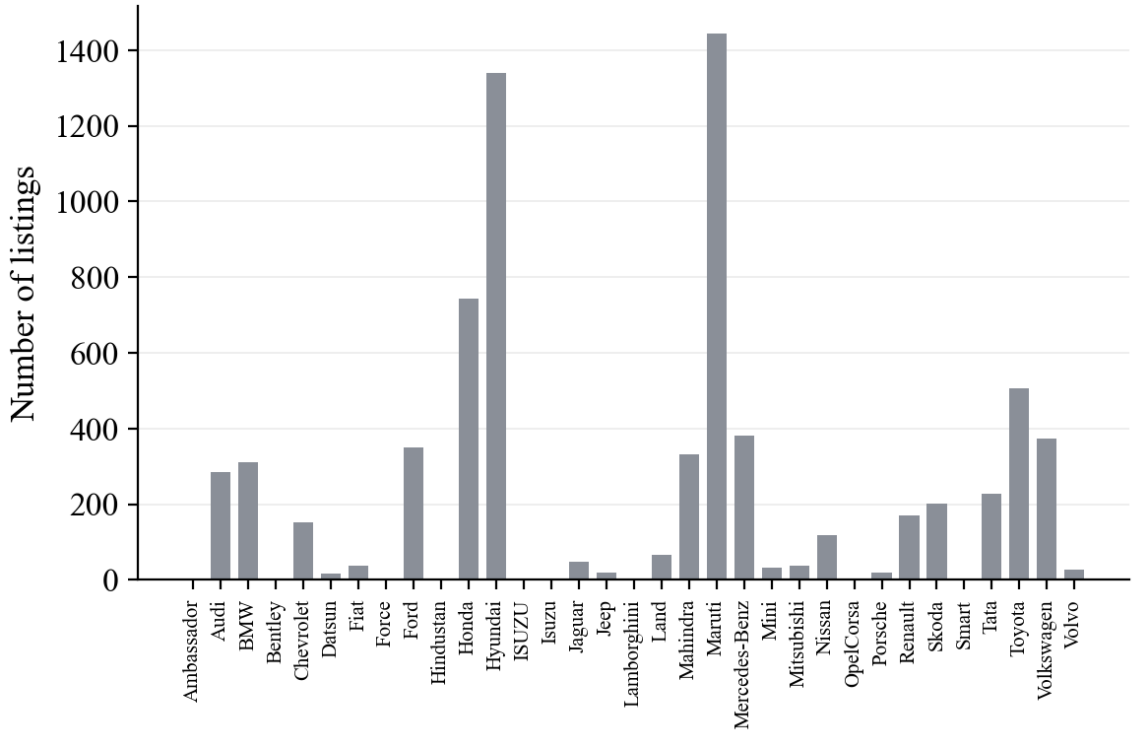
- **Same 33 brands, same counts. The only differences are orientation and order.**

Which brands are in this dataset? Same data, twice the reading time

Sorted, horizontal, top 15




Alphabetical, vertical, all 33 brands



# Act 2, Closed

- **Four ways to show an amount, and when each one wins**
  - Bar — the default. Sort by value, go horizontal when labels are long.
  - Grouped bar — comparing within a category. Both series need the same baseline.
  - Dot plot — when no value is near zero and bars would waste the axis.
  - Heatmap — when the grid exceeds roughly 5 x 5.
- **None of this changed a cleaning decision. That is fine — this act was orientation.**

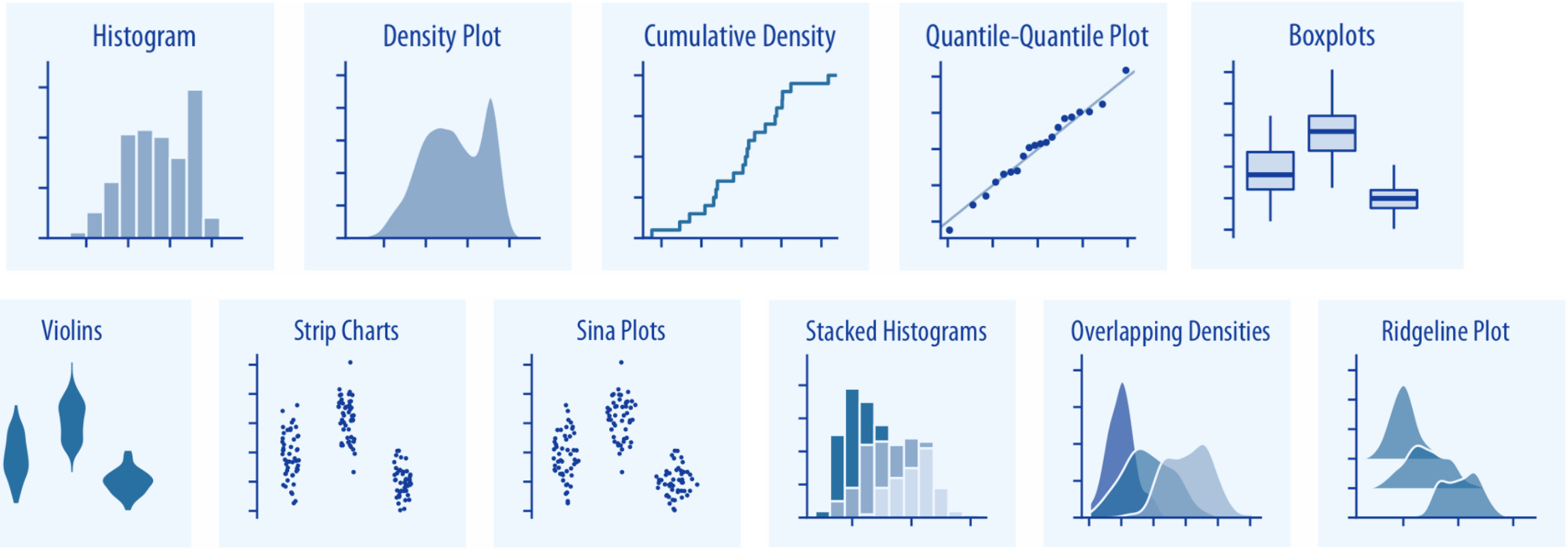
# Univariate: Numeric Variables

Structure – Clean – **Univariate** – Bivariate – Multivariate – Decisions

## ➢ ACT 3

- **What shape is each variable, and the target above all?**

# Directory of Visualizations: Distribution




Source: Fundamentals of Data Visualization,
Claus O. Wilke, O'Reilly Media 2019

# Introduction to Distribution

- Common methods of visualizing distributions
  - **Histograms**: **Frequency** of data in different bins or ranges.
  - **Density Plots**: **Probability distribution** of data over a **continuous** interval or time period.
  - **Cumulative Densities**: **Cumulative probability** of data up to a certain value.
  - **q-q (Quantile-Quantile) Plots**: Compare the quantiles of the dataset to the quantiles of a normal distribution to **check for data normality** or other distributional assumptions.
  - **Box Plots:** **Spread** of data through quartiles and can **highlight outliers**.
  - **Violin Plots**: Combine aspects of **box plots and density plots** to show the distribution, probability density of continuous data.

# Histogram: Essential Points

- **Definition:** Represents the **distribution** of a variable by dividing the data into bins (intervals) and **counting the number of observations** that fall within each bin. The height of each bar shows **the frequency of observations** in each bin (**sns.histplot() or plt.hist()**).
- **Key features**
  - **Discrete or grouped continuous data**: Histograms are usually useful for both discrete data and grouped **continuous data**.
  - **Bins**: The choice of **bin size / number** can significantly affect the appearance of the histogram.
    - **Too many bins** can make the data look **noisy**.
    - **Too few bins** can **oversimplify** the distribution.
  - **Frequency or Density**: Histograms can be scaled to show **frequencies** (**the count of observations** in each bin) or **density** (**the proportion of observations** per unit of the variable).

# Density Plot: Essential Points

- Definition: A **smoothed, continuous** line **estimated from the data** representing the distribution of the data. Often through a method known as **kernel density estimation (KDE) (sns.kdeplot())**.
- **Key features**
  - **Smoothness**: Density plots provide **a smooth curve** that represents the **distribution.**
  - **Kernel Density Estimation (KDE):** Used to create **a smooth estimate** of the distribution.
  - **Area Under the Curve:** The total area under the curve is usually **normalized to 1**.

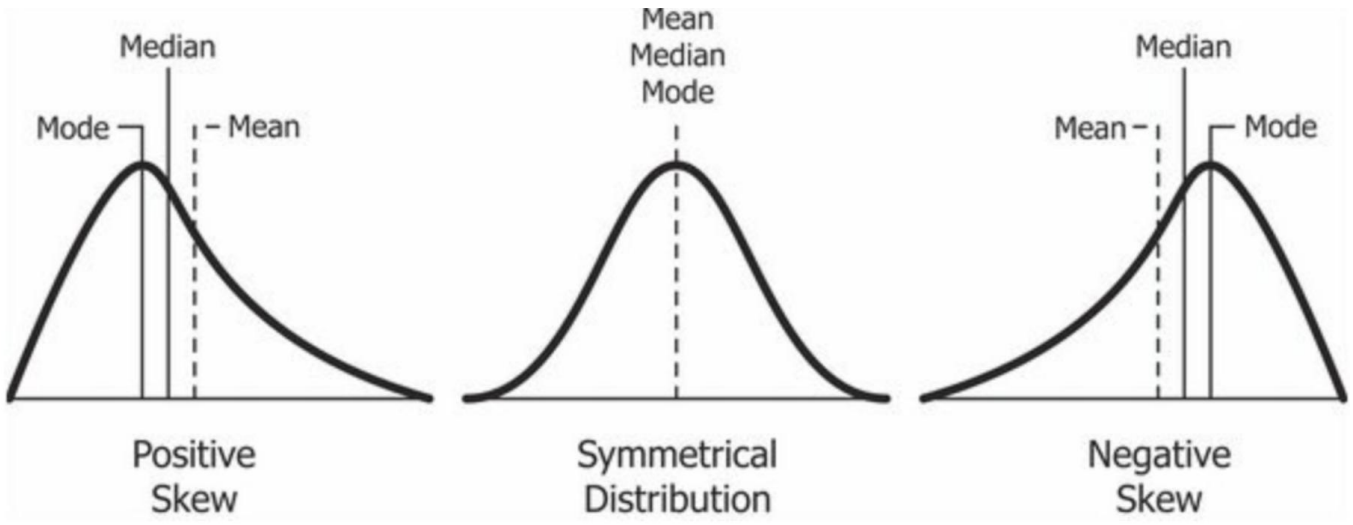
# Distribution Analysis

- **Spread:**
  - **Range**: The difference between the maximum and minimum data points.
  - **Variance & Standard Deviation**: Measures of the spread of data points around the mean.
- **Peaks:**
  - **Mode**: The value or values that appear most often. A histogram can be
    - **Unimodal**: One peak
    - **Bimodal**: Two peaks
    - **Multimodal**: More than two peaks

# Distribution Analysis

## ➢ Shape:

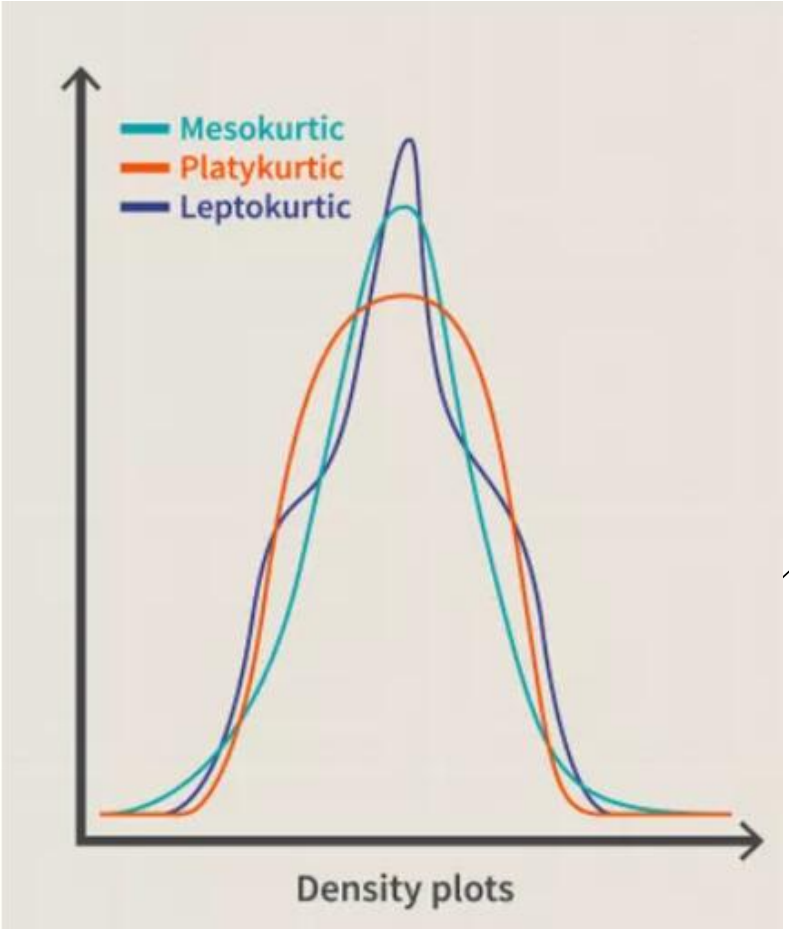
- **Skewness**: Is the histogram symmetrical or skewed? (**function: stats.skew()**)
  - **Symmetrical**: Looks the same on both sides of the center; Mean = Median = Mode
  - **Skewed Right (Positive Skew)**: Tail is longer on the right side; Mean > Median > Mode
  - **Skewed Left (Negative Skew)**: Tail is longer on the left side; Mean < Median < Mode



# Distribution Analysis

## ➢ Kurtosis

- **Leptokurtic**: Has a **sharp peak** and **heavy tails**, indicating a lot of data near the mean but also frequent large deviations.
- **Mesokurtic**: normal distribution shape.
- **Platykurtic**: Has **a flatter peak** and **thinner tails**, indicating data is spread out from the mean.
- Function: **stats.kurtosis()**
  - Mesokurtic: near zero
  - Leptokurtic: >0
  - Platykurtic: <0



# Distribution Analysis

- **Outliers:**
  - Look for bars that are detached from the main body of the histogram or are unusually low or high, which might indicate outlier data points.
- **Gaps:**
  - Significant spaces between bars might indicate a lack of data in those ranges or could indicate data clusters.
- **Volume of Data:**
  - A histogram with a large sample size may appear smoother, whereas one with a smaller sample size might appear more jagged or rough.

# Quartiles, Interquartile Range (IQR) and Outliers

- Quartiles: Divide an ascending dataset into **four equal parts**, helping to provide a summary of the spread and center of the data
  - **Q1 (First Quartile)** represents the median of the lower half of the dataset.
  - **Q2 (Second Quartile or Median)** is the median of the entire dataset.
  - **Q3 (Third Quartile)** represents the median of the upper half of the dataset.
- **Interquartile Range (IQR):** IQR=Q3-Q1
- **Outliers:**
  - **Lower Bound (for outliers)**: $Q1-1.5\times IQR$
  - **Upper Bound (for outliers)**: $Q3+1.5\times IQR$

# Box Plot

- A graphical representation of the **distribution** of a dataset. Providing a **summary** of a set of data values: **sns.boxplot() or plt.boxplot()**
- **Components of a Box Plot:**
  - **Central Box:**
    - The bottom line of the box: Q1 (25th percentile).
    - The middle line of the box: Median (50th percentile).
    - The top line of the box: Q3 (75th percentile).
    - Represents the interquartile range (IQR).

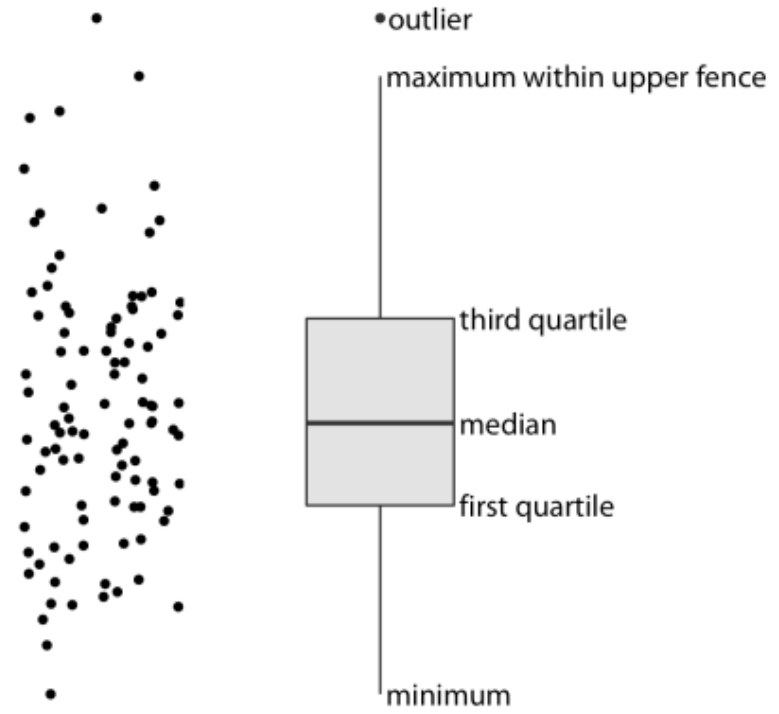



Figure 9-2. Anatomy of a boxplot. Shown are a cloud of points (left) and the corresponding boxplot (right).

# Box Plot

➢ **Components of a Box Plot:**

- **Whiskers:**
  - Indicate variability outside the upper and lower quartiles.
  - **Lower whisker**: Minimum data point within Q1−1.5×IQR.
  - **Upper whisker**: Maximum data point within Q3+1.5×IQR.
- **Outliers:**
  - Data points outside the whiskers.
  - Represented as dots.

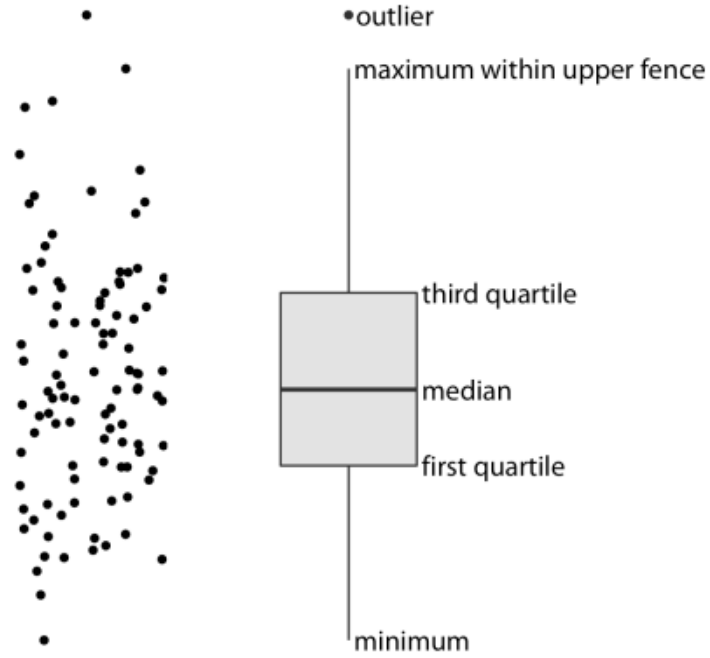
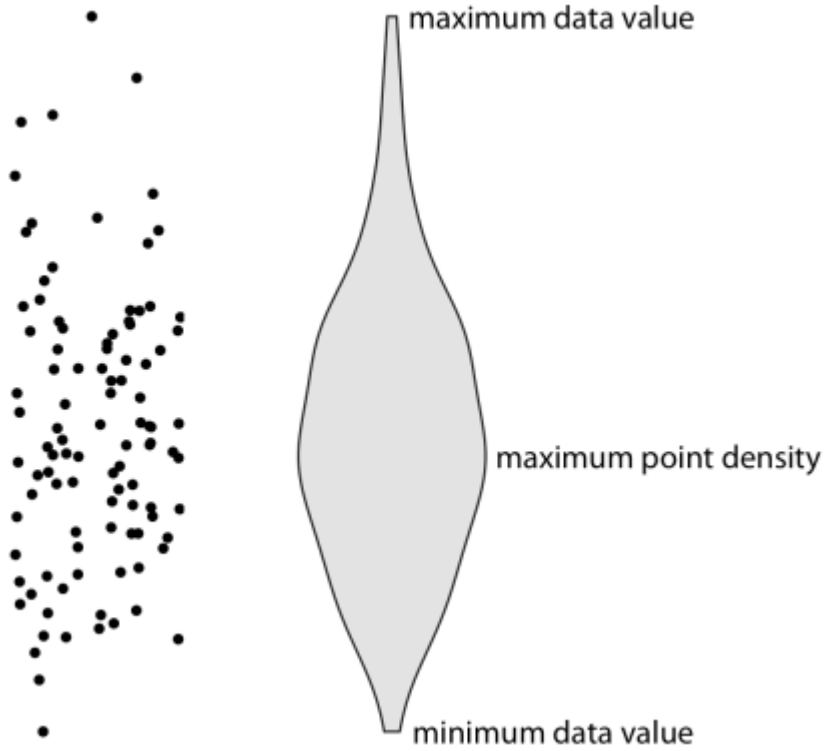


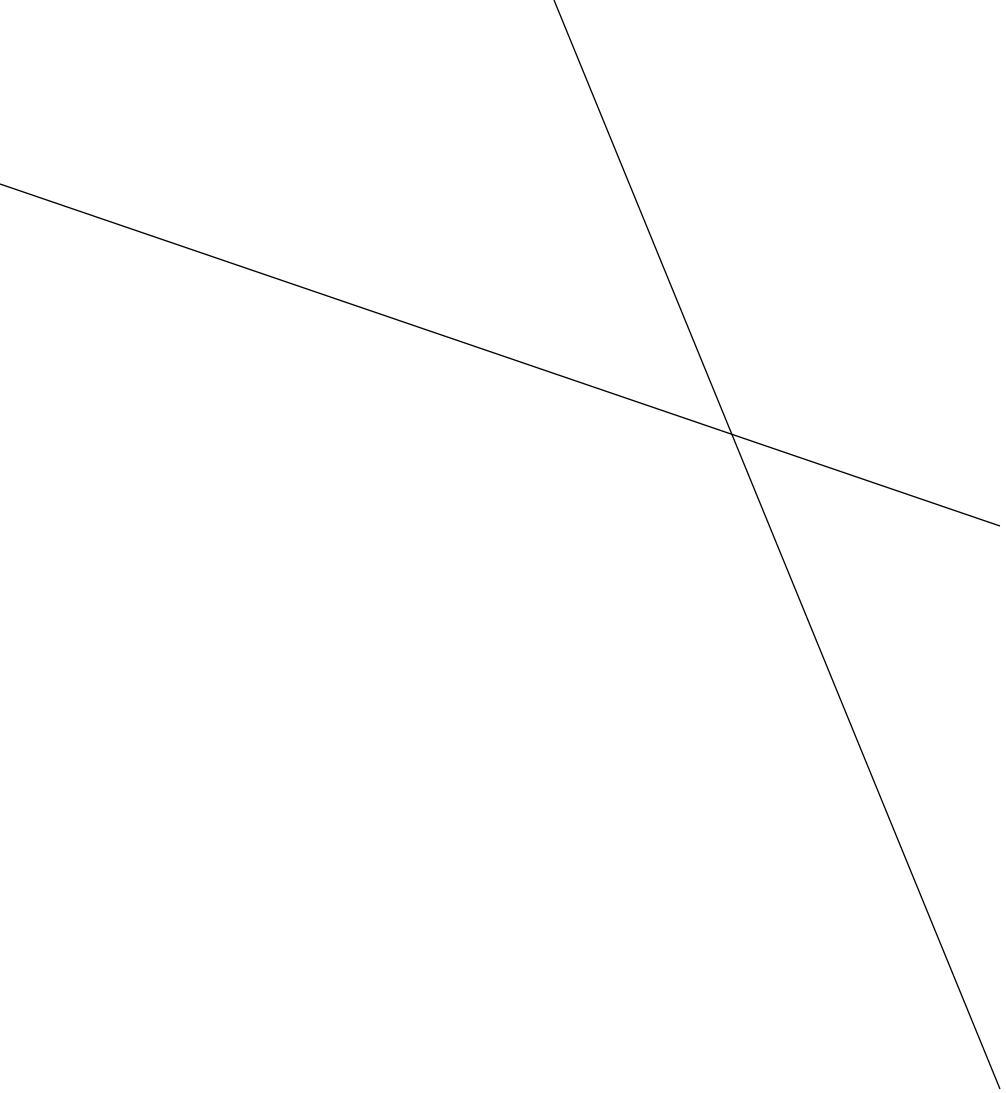

Figure 9-2. Anatomy of a boxplot. Shown are a cloud of points (left) and the corresponding boxplot (right).

# Violin Plot

- A combination of **a box plot** and a **kernel density plot: sns.violinplot()**.
- Technically, a violin plot is a density estimate rotated by 90 degrees and then mirrored.
- Violins are **symmetric**
- Violins begin and end at the minimum and maximum data values
- The thickest part of the violin corresponds to the **highest point density** in the dataset.




- Figure 9-4. Anatomy of a violin plot. Shown are a cloud of points (left) and the corresponding violin plot (right)



# THANK YOU